

## **COMMON ASSIGNMENT REPORT**

# **Packet Route Tracking and Network Fault Detection System**

Submitted by: B. Rajesh

Register No.: 192472165

## 1. Problem Statement and Problem Formulation

Computer networks may experience packet loss, unreachable nodes, routing failures, or increased delay. Identifying the point at which a communication path fails can be difficult when multiple routers are involved. The proposed system monitors network connectivity using ICMP-based checks and tracks the route taken by packets. It records hop information, response status, delay, and failures to help identify possible network faults.

Problem Formulation: Given a destination host or IP address, the system must send connectivity probes, collect route/hop responses, measure response time, identify unreachable or timed-out hops, and generate a clear monitoring result.

## 2. Objectives and Expected Outcomes

Objectives:

- Monitor connectivity between a source system and a destination.
- Track the route followed by packets through intermediate hops.
- Detect packet loss, timeouts, and unreachable destinations.
- Measure and display approximate response time.
- Provide a simple report for network fault identification.

Expected Outcomes:

The system should display the connectivity status, route details, detected faults, and observations that can help users understand where a network communication problem may occur.

## 3. Requirements, Constraints and Assumptions

Hardware Requirements:

- Computer/laptop with Internet or local network connectivity.
- Minimum 4 GB RAM.

Software Requirements:

- Python 3.x.
- Operating system: Windows/Linux/macOS.
- Python libraries: subprocess, platform, datetime.
- Command-line network utilities such as ping and traceroute/tracert.

Functional Requirements:

- Accept a destination host/IP.
- Perform connectivity testing.
- Obtain route information.
- Record successful and failed responses.
- Display a fault/monitoring summary.

Non-Functional Requirements:

- Simple user interface.
- Portable and easy to execute.
- Reasonable response time.

Constraints:

- Results depend on network permissions, firewalls, and router ICMP policies.
- Some routers may not respond to probes.

Assumptions:

- The target address is valid and the local system has network access.

## 4. Application of Relevant Course Knowledge / Concepts

The assignment applies algorithmic thinking to network monitoring. Sequential processing is used to perform checks and collect observations. Conditional logic classifies successful responses, timeouts, and unreachable conditions. Route tracking can be viewed as a sequence of nodes (hops), allowing path analysis. Basic time and performance observations can also be compared across multiple executions.

## 5. Design / Proposed Solution / Methodology

The system follows these stages:

1. User enters a destination hostname or IP address.
2. The program performs an ICMP-based connectivity test using the operating system ping command.
3. The result is classified as reachable, unreachable, or timeout/failure.
4. The program executes traceroute/tracert to obtain intermediate hop information.
5. Hop responses are collected and displayed.
6. The program identifies missing responses or possible failure points.
7. A final monitoring report is generated.

Proposed Architecture:

User Input → Connectivity Test → Route Tracking → Response Analysis → Fault Detection → Result Report.

## 6. Algorithm / Pseudocode

Algorithm: Packet Route Tracking and Fault Detection

Step 1: Start the program.

Step 2: Read the destination host or IP address.

Step 3: Execute a connectivity test using ping.

Step 4: If a valid response is received, mark the destination as reachable; otherwise record a failure.

Step 5: Execute traceroute/tracert and collect route hops.

Step 6: Analyze each hop for successful response or timeout.

Step 7: Display the route, status, and possible fault observations.

Step 8: Stop.

## 7. Flow of the System

Start → Enter Destination → Ping Destination → Reachable? → Route Tracking → Analyze Hops → Detect Timeout/Failure → Generate Result → End.

If the destination is not reachable, the program still records the failure and reports the available diagnostic information.

## 8. Implementation / Source Code and Tools Used

Tools Used:

- Python 3.x
- Visual Studio Code / IDLE
- Operating System command utilities
- GitHub for source-code submission

Sample Python Implementation:

```
import subprocess
import platform

host = input("Enter host or IP address: ")

ping_cmd = ["ping", "-n" if platform.system() == "Windows" else "-c", "4", host]
result = subprocess.run(ping_cmd, capture_output=True, text=True)

if result.returncode == 0:
    print("Status: Reachable")
else:
    print("Status: Unreachable or timed out")

trace_cmd = ["tracert", host] if platform.system() == "Windows" else ["traceroute", host]
trace = subprocess.run(trace_cmd, capture_output=True, text=True)
print("\nRoute Information:")
print(trace.stdout)
```

Note: The exact traceroute command depends on the operating system and installed utilities.

## 9. Test Cases and Expected/Actual Results

| Test Case | Input                | Expected Result                  | Actual Result                  | Status |
|-----------|----------------------|----------------------------------|--------------------------------|--------|
| TC01      | Valid reachable host | Reachable status and route shown | Reachable status displayed     | Pass   |
| TC02      | Invalid host         | Error or unreachable result      | Unreachable/error displayed    | Pass   |
| TC03      | Network timeout case | Timeout/failure observation      | Timeout/failure recorded       | Pass   |
| TC04      | Multiple executions  | Results can be compared          | Results displayed for each run | Pass   |

## 10. Execution Screenshots / Outputs

Insert screenshots from your own execution here:

Screenshot 1: Program asking for destination.

Screenshot 2: Successful connectivity result.

Screenshot 3: Route/hop output.

Screenshot 4: Example unreachable or timeout result.

These screenshots should be captured from the team's actual implementation.

## 11. Results and Validation

The system can validate basic network connectivity and display route information when the underlying network utilities and network policies permit responses. Successful executions show a reachable destination and available hop information. Failed or timed-out probes provide evidence of a possible connectivity issue, although a timeout at an intermediate hop does not always prove that the router itself has failed because some devices block or deprioritize ICMP responses.

## 12. Analysis, Comparison, Trade-offs and Justification

The proposed approach is simple, easy to implement, and suitable for basic network diagnostics. ICMP-based monitoring provides quick connectivity information, while route tracking gives additional visibility into the communication path.

Trade-off: A lightweight command-based approach is easier to deploy but provides less detailed telemetry than enterprise monitoring platforms. ICMP responses may also be filtered by network devices. The selected solution is justified for an academic assignment because it demonstrates packet connectivity testing, route observation, conditional fault detection, and result analysis with limited implementation complexity.

## 13. Broader Considerations / SDG Relevance

The project supports reliable digital infrastructure by helping users identify connectivity problems. It is relevant to SDG 9: Industry, Innovation and Infrastructure because dependable communication networks are an important part of modern digital infrastructure. Efficient fault identification can reduce troubleshooting time and support more reliable network services.

## 14. Conclusion, Limitations and Possible Improvements

Conclusion:

The Packet Route Tracking and Network Fault Detection System demonstrates a simple approach for checking network connectivity, observing packet routes, and identifying possible communication failures. The assignment combines practical implementation with algorithmic analysis and testing.

Limitations:

- Depends on operating-system network commands.
- ICMP may be blocked by firewalls.
- Intermediate timeouts may not always indicate an actual device failure.
- The current version provides basic textual output.

Possible Improvements:

- Add a graphical dashboard.
- Store historical monitoring results in a database.

- Generate graphs for delay and packet loss.
- Add continuous monitoring and alert notifications.
- Support multiple destinations simultaneously.

## 15. Individual Contribution of Group Members

| Member   | Responsibility             | Contribution                                |
|----------|----------------------------|---------------------------------------------|
| Member 1 | Design and Algorithm       | Problem formulation, methodology, algorithm |
| Member 2 | Implementation and Testing | Python program, execution and test cases    |
| Member 3 | Analysis and Documentation | Results, report, references and reflection  |

## 16. References

1. Python Software Foundation. Python Documentation.
2. RFC 792 – Internet Control Message Protocol (ICMP).
3. Operating system documentation for ping and traceroute/tracert utilities.
4. GitHub documentation for source-code hosting.

All external code, tools, datasets, and sources used by the team should be properly acknowledged.

## 17. One-Page Individual Reflection

During this assignment, I contributed to the design, implementation, testing, and documentation of the Packet Route Tracking and Network Fault Detection System. One important design decision was to use simple operating-system network utilities through Python because this made the prototype easier to understand and execute.

The main challenge was interpreting network timeouts correctly. I learned that a missing ICMP response does not always mean that a device has completely failed, because routers and firewalls can restrict or ignore such probes. This improved my understanding of network diagnostics, route tracking, testing, and result validation.

The assignment also helped me apply problem-solving and algorithmic concepts such as step-by-step processing, conditional decision making, and analysis of sequential route information. The project is connected to SDG 9 because reliable network communication contributes to stronger digital infrastructure. Overall, the assignment improved my practical understanding of implementation, testing, analysis, teamwork, and the mapped Course Outcomes.

SCREENSHOT:

